TRADUZIONE DEL SECONDO CAPITOLO (PARTE 2)

Logical and Algorithmic Thinking

PENSIERO ALGORITMICO

Ora passiamo dal pensiero logico al pensiero algoritmico. Questa sezione introdurrà le proprietà di base degli algoritmi (sequenza, iterazione e selezione), nonché l'uso dello stato.

In breve: Algoritmi

Logica e algoritmi non sono la stessa cosa. Piuttosto, gli algoritmi si basano sulla logica perché, come parte del loro lavoro, prendono decisioni logiche. L'altra parte del loro lavoro è "cucire" insieme quelle decisioni.

La logica ti fornisce un insieme di regole che ti consentono di ragionare su un certo aspetto del mondo. Quel mondo non deve essere statico. La logica può occuparsi di cose dinamiche e in continuo cambiamento. Questo lo abbiamo visto con gli esempi di Tris. Le regole del gioco sono state stabilite come affermazioni logiche. A volte le proposizioni erano vere, a volte false. Elaborando i loro valori di verità, potevamo giungere a conclusioni su come sarebbe stato il gioco in diverse situazioni.

Ma se vogliamo andare oltre, se vogliamo costruire sistemi funzionanti basati su regole, allora la sola logica non è sufficiente. Abbiamo bisogno di qualcosa che possa integrare tutte queste regole ed eseguire azioni basate sui risultati della loro valutazione. Quel qualcosa sono gli algoritmi e sono il potere dietro tutti i sistemi computazionali del mondo reale.

Algoritmo: una sequenza di passaggi chiaramente definiti che descrivono un processo da seguire, un insieme finito di istruzioni non ambigue con chiari punti di inizio e fine.

Gli algoritmi sono un modo per specificare un'attività in più fasi e sono particolarmente utili quando desideriamo spiegare a una terza parte (che sia umana o macchina) come eseguire i passaggi con estrema precisione. Come con la logica, gli umani hanno già una comprensione intuitiva degli algoritmi. Ma, allo stesso tempo, una scienza ricca e precisa detta esattamente come funzionano gli algoritmi. Acquisire una comprensione più profonda di ciò migliorerà il tuo pensiero algoritmico. Questo è importante perché un algoritmo corretto è la base fondamentale di qualsiasi soluzione basata su computer.

Intuizione vs precisione

Gli algoritmi sono cose complicate. In un certo senso, sono concetti intuitivi che sono con noi da secoli, eppure hanno ricevuto una definizione solo negli ultimi cento anni circa (Knuth, 1997). È facile articolarli intuitivamente, ma allo stesso tempo il concetto di algoritmo dell'informatica è многосfaccettato e può richiedere ai principianti un po' di tempo per comprenderlo. Comunemente sorgono incomprensioni (vedi Pea et al., 1987; Pane et al., 2001), quindi ci concentreremo sulla costruzione di una solida comprensione.

Prima di entrare nei dettagli difficili, prendiamo come punto di partenza la semplice definizione nel riquadro sopra. Molto probabilmente hai già avuto a che fare con algoritmi in questo senso. Chiunque abbia seguito una ricetta mentre cucinava ha incontrato qualcosa di simile. O chiunque sia andato a una caccia al tesoro da bambino. O chiunque abbia assemblato mobili. Un algoritmo è

un modo per rendere espliciti i nostri processi in modo da poterli comunicare a qualcun altro in un modo che gli consenta di eseguire anche quei passaggi. Queste sono solo analogie, tuttavia. In senso stretto, gli algoritmi eseguono operazioni sui dati piuttosto che sugli ingredienti della torta o sui tavolini da caffè.

I primi pionieri dell'informatica hanno esaminato cosa significasse comunicare idee a un computer. Cercavano un mezzo per noi per prendere le idee nelle nostre teste e metterle in una forma che i computer potessero capire ed elaborare per nostro conto. Hanno scoperto che le nostre idee esistenti e intuitive di algoritmi erano insufficienti, quindi hanno preso quelle idee e le hanno riformate nel tipo di forma chiara e non ambigua che i computer richiedono. In tal modo, hanno reso gli algoritmi il nostro mezzo per dare istruzioni ai computer.

Per quanto precisi siano diventati, gli algoritmi sono stati anche resi più complessi, tanto che una descrizione formale di essi può estendersi su diversi paragrafi. Nel nostro caso, il modo migliore per affrontare una definizione è prendere ciascuna proprietà degli algoritmi, una per una, e spiegarla. Lungo il percorso, useremo un'analogia con una delle nostre idee intuitive di un algoritmo (in questo caso, seguire una ricetta) per aiutarci a capire.

Definizione di algoritmi

La definizione di un algoritmo è complessa e coinvolge diverse proprietà. Questa sottosezione descrive tali proprietà.

Raccolta di singoli passaggi

La prima proprietà da menzionare ribadisce qualcosa detto in precedenza: un algoritmo è una raccolta di singoli passaggi. Una ricetta si adatta a questa analogia abbastanza semplicemente, piena com'è di passaggi come: "preriscalda il forno a 180 gradi Celsius" o "aggiungi due cucchiai di zucchero alla ciotola".

Definitezza

A seguire da questa proprietà c'è la definitezza, il che significa che ogni passaggio deve essere definito con precisione. Ogni passaggio in un algoritmo può avere uno e un solo significato, altrimenti è ambiguo. Allo stesso modo, gli chef sono giunti alla stessa conclusione, motivo per cui producono ricette usando misurazioni precise invece di scrivere cose come "un po' di zucchero" o "cuocere per un po'".

Sequenziale

Gli algoritmi sono anche sequenziali. I passaggi che compongono il processo devono essere eseguiti nell'ordine specificato. Mancare di farlo significa che il risultato dell'esecuzione dell'algoritmo è probabilmente errato. Ripensa all'analogia. Tagliare a dadini una cipolla e friggere una cipolla sono passaggi diversi. Tagliare a dadini una cipolla prima di friggerla ha un risultato diverso rispetto al contrario. Allo stesso modo, moltiplicare un numero per 2 e poi aggiungere 5 ad esso produce un risultato diverso dall'aggiungere prima 5 e poi raddoppiarlo. Come una ricetta, devi rispettare la sequenza quando esegui un algoritmo affinché abbia un risultato significativo.

Deviazione: Stato negli algoritmi

Vale la pena fare una breve deviazione qui per esaminare perché la sequenza è così importante per gli algoritmi. Ha tutto a che fare con lo stato, con il quale intendo semplicemente i valori correnti di tutte le cose di cui l'algoritmo tiene traccia. Man mano che un computer avanza attraverso un algoritmo, proprio come tu avanzi attraverso una ricetta, lo stato delle cose può cambiare. Chiaramente, sequenziare i passaggi di un algoritmo assicura che lo stato cambi sempre nello stesso modo ogni volta che l'algoritmo viene eseguito.

Stato: la configurazione corrente di tutte le informazioni di cui un programma tiene traccia in un qualsiasi istante nel tempo.

C'è un'importante implicazione in questo: non esiste una "visione globale" quando si tratta di algoritmi. Ciò significa che, in ogni istante nel tempo, l'ambiente in cui l'algoritmo viene eseguito esiste in un determinato stato. Ma nel momento in cui viene eseguito il passaggio successivo, qualcosa potrebbe essere cambiato. L'ambiente esiste realmente come una serie di istantanee, una per ogni passaggio dell'algoritmo. L'analogia della ricetta lo spiega bene. All'inizio potresti avere burro, farina, latte, uova e zucchero. Dopo ogni passaggio, scatti una fotografia della cucina. Le foto mostreranno che, poco a poco, lo stato degli ingredienti cambia. La farina va in una ciotola; poi si uniscono le uova; poi il burro va nella padella; e così via. Non esiste una visione globale degli ingredienti; solo una serie di istantanee.

Per gli algoritmi, questo significa che i singoli passaggi vengono eseguiti uno per uno e solo un singolo passaggio può essere preso in considerazione in un dato momento. Una volta che un passaggio è stato eseguito, il computer se ne dimentica e passa al successivo.

Ciò significa che deve essere fornito un modo per "ricordare" le cose accadute nei passaggi precedenti. Se tu, l'autore dell'algoritmo, vuoi che il computer ricordi il risultato di un passaggio per un uso successivo, devi dirglielo esplicitamente. Gli algoritmi lo prevedono per mezzo di variabili.

Pur condividendo lo stesso nome, le variabili nella programmazione non corrispondono esattamente alle variabili della matematica. Le variabili matematiche vengono utilizzate in una funzione per rappresentare quantità che variano. Una variabile in un algoritmo è più simile a un blocco note utilizzato come segnaposto per informazioni importanti di cui il computer dovrebbe tenere nota. Può anche avere i suoi valori aggiornati durante l'esecuzione di un algoritmo (questo si chiama assegnamento). Ad esempio, se il tuo algoritmo conta il numero di volte in cui una parola specifica appare in un articolo di notizie, utilizzerebbe una variabile per tenere traccia del numero mentre legge il testo.

Controllo dell'esecuzione dell'algoritmo

In questa sezione esamineremo due modi per controllare l'esecuzione di un algoritmo.

Iterazione

Oltre a registrare informazioni pertinenti a un problema, le variabili possono anche essere utilizzate per controllare l'esecuzione di un algoritmo. A un livello base, possono essere utilizzate in due modi. Uno di questi è l'iterazione, nota anche come ciclo. L'iterazione consente di ripetere una serie di passaggi più e più volte, senza la necessità di scrivere manualmente ogni singolo passaggio. Questo può essere molto utile. Immagina di dover scrivere il testo di una canzone come "99 Bottles of Beer". Se non la conosci, fa così:

99 bottles of beer on the wall, 99 bottles of beer. Take one down, pass it around, 98 bottles of beer on the wall.

98 bottles of beer on the wall, 98 bottles of beer. Take one down, pass it around, 97 bottles of beer on the wall.

E continua a ripetersi fino all'ultima strofa:

1 bottle of beer on the wall, 1 bottle of beer. Take it down, pass it around, No more bottles of beer on the wall.

Per risparmiarti un sacco di battiture, potresti invece scrivere solo una strofa insieme alle istruzioni su come ripetere tutte le strofe successive. Qualcosa del genere:

X rappresenta il numero di bottiglie. All'inizio, X è 99. Canta la strofa:

X bottles of beer on the wall, X bottles of beer. Take one down, pass it around, X-1 bottles of beer on the wall.

Sottrai 1 da X. Ripeti la strofa se X è maggiore di 0, altrimenti termina.

Questo è un esempio di iterazione in azione e mostra due cose:

1. Come una variabile viene utilizzata per controllare l'esecuzione dell'algoritmo. La cosa che cambia da un ciclo all'altro è incapsulata in una variabile, in questo caso il numero di bottiglie di birra rimanenti. Tutto il resto rimane costante.
2. Devi specificare in quali condizioni il ciclo deve terminare.

Quest'ultimo punto ci porta al secondo metodo utilizzato per controllare l'esecuzione dell'algoritmo.

Selezione

In un ciclo, un modo per controllare quante volte i passaggi vengono ripetuti è semplicemente specificarlo; qualcosa come "ripeti i seguenti passaggi 99 volte". Ma nota che l'esempio precedente non lo fa. Invece, usa la selezione (nota anche come condizionale), che è un modo per testare il valore corrente di una variabile e prendere una decisione in base ad esso. Nella canzone "99 Bottles", il condizionale arriva alla fine (ripeti la strofa se X è maggiore di 0). Dice al computer di fare qualcosa finché la condizione specificata è attualmente vera. Finché rimane un numero positivo di bottiglie, tale condizione è vera e quindi la strofa viene ripetuta. Ma sappiamo che il numero diminuisce costantemente e alla fine raggiungerà lo zero. A questo punto la condizione diventa falsa (zero non è maggiore di zero) e quindi la strofa non viene ripetuta di nuovo.

Le condizioni possono essere utilizzate in qualsiasi fase di un algoritmo, non solo per controllare i cicli. Ovunque si trovino, servono tutti allo stesso scopo: creare un punto in cui il computer deve decidere se eseguire una serie di passaggi o meno, e incapsulare le informazioni necessarie per prendere tale decisione.

Esempio di algoritmo

Questo esempio illustra alcune delle idee appena discusse. È un algoritmo parziale e molto semplice per controllare un gioco di Tris. È scritto usando pseudocodice, che è un mezzo informale di

scrittura di un algoritmo. È rivolto a un pubblico umano piuttosto che a un computer, ma segue comunque da vicino le convenzioni dei linguaggi di programmazione. Ciò significa che puoi facilmente tradurre le tue idee in codice di programma reale scrivendole prima come pseudocodice.

1. il gioco è iniziato
2. inizia ciclo:
3. chiedi al giocatore di scegliere una casella
4. se la casella scelta non è occupata, allora metti il simbolo del giocatore in quella casella
5. controlla la griglia per vedere se è stata realizzata una riga
6. se è stata realizzata una riga, allora il gioco è vinto
7. se non è stata realizzata una riga e non sono disponibili caselle, allora il gioco è patto
8. passa all'altro giocatore
9. esci dal ciclo se il gioco è vinto o il gioco è patto
10. visualizza il messaggio "Game over"

Esaminiamo questo algoritmo:

- Quando questo algoritmo viene eseguito, il computer passa attraverso ogni riga una alla volta. (Questo è un esempio di sequenza.)
- La riga 1 inizializza una variabile, *game*, con un valore particolare, *started* (stato).
- La riga 2 imposta il punto di partenza di un ciclo (iterazione). Tutte le righe che sono "dentro" questo ciclo sono indentate per rendere più chiaro ciò che è dentro e fuori.
- La riga 3 chiede input al giocatore. La scelta viene registrata.
- La riga 4 effettua una selezione in base alla scelta del giocatore. (Per inciso, questo è un algoritmo piuttosto poco user-friendly. Se un giocatore sceglie accidentalmente una casella occupata, non ha l'opportunità di correggere il suo errore!)
- Le righe 6 e 7 effettuano entrambe delle selezioni, alterando il valore della variabile *game* sotto determinate condizioni.
- La riga 9 è la fine del ciclo. Viene fatta una scelta se ripetere o meno il ciclo a seconda della condizione associata a questo punto. Ci sono due possibilità:
    1. La variabile *game* non è stata ancora alterata, il che significa che ha ancora il valore *started*. In questo caso, l'esecuzione torna alla riga 3.
    2. A un certo punto, il gioco è stato vinto o pareggiato. In entrambi i casi, il ciclo termina e l'esecuzione continua alla riga 10, dopodiché il gioco termina con un tradizionale messaggio "Game over".

"GOTCHAS" (Insidie)

Quando si incontrano per la prima volta la logica e gli algoritmi, la maggior parte delle persone tende a commettere gli stessi errori o presupposti errati. Questa sezione li indicherà e spiegherà come evitarli.

La necessità di chiarezza e meticolosità

Potresti aver notato l'enfasi sulla chiarezza, la precisione e la meticolosità quando si lavora con la logica e gli algoritmi. I programmi per computer richiedono precisione nelle loro regole e istruzioni a tal punto che potrebbe sembrare pedanteria. Ma in realtà non lo è. Ci sono buone ragioni dietro la necessità di chiarezza, che si riducono tutte alla natura di un computer.

Vale la pena sottolineare un paio di fatti al riguardo:

- Un computer farà esattamente ciò che gli viene detto. Se gli viene detto di fare qualcosa di impossibile, si bloccherà.
- Un computer non ha intelligenza innata propria. Non farà nulla che non gli sia stato istruito di fare.
- Un computer non ha buon senso. Non cercherà di interpretare le tue istruzioni in modi diversi. Non farà supposizioni o riempirà gli spazi vuoti ovvi in un algoritmo incompleto.

In breve, un computer farà solo ciò che gli dici di fare. Se non gli dici di fare qualcosa, non lo farà. Se dici a un computer di fare qualcosa di stupido, come dividere un numero per zero, cercherà di farlo comunque. Se un computer fa qualcosa che non pensavi di avergli detto di fare, probabilmente è perché hai frainteso le tue stesse istruzioni.

Prima, ho fatto un'analogia tra un algoritmo e una ricetta. Tuttavia, il pubblico di una ricetta è un'altra persona. Certo, una persona può essere il pubblico di un algoritmo, ma le persone sono abituate a comunicare usando il linguaggio naturale. Anche quando è molto formale, quel linguaggio può comunque essere ambiguo e a volte aperto all'interpretazione. Inoltre, l'autore può lasciare certe cose al buon senso di un lettore piuttosto che insistere su un punto.

Devi essere consapevole dei limiti (elettronici) del computer delineati sopra. Pensa di dare una ricetta a un robot estremamente letterale di un film di fantascienza scadente. Sarebbe colpa del robot se leggesse "mescola latte, farina e uova in una ciotola" e poi cercasse di arrampicarsi nella ciotola stessa? O se aspettasse per sempre che "l'impasto lieviti" perché l'impasto non ha ancora iniziato a levitare?

Il punto è che l'obiettivo della risoluzione dei problemi usando il pensiero computazionale è una soluzione che un computer possa eseguire. Ciò significa un algoritmo con un significato preciso e non ambiguo che non richieda né intelligenza creativa né buon senso per essere compreso appieno.

E questo spesso significa spaccare il capello e spiegare le cose in dettagli apparentemente pedanti.

Uso errato degli operatori logici

La nostra comprensione intuitiva del linguaggio porta ad altre "insidie". Qualcosa di apparentemente semplice come il nostro uso delle parole "e", "o", "non" e "allora" può coglierci di sorpresa. Ad esempio, l'affermazione:

"Tutti coloro il cui cognome inizia con A e B sono assegnati al Gruppo 1."

potrebbe sembrare logica a un essere umano, ma per un computer, che segue rigorosamente il significato logico di AND, è illogica. Questo perché, così come è scritta, l'affermazione verifica due cose: il cognome inizia con A e inizia con B? Tuttavia, nessun cognome può avere lettere iniziali diverse contemporaneamente. Puoi chiedere a un computer di eseguire questo test quanto vuoi, ma fallirà sempre.

Inoltre, nel linguaggio di tutti i giorni, le persone a volte usano "e" come termine di sequenziamento, come in:

"Il giocatore ha fatto il suo turno e il gioco è finito."

Di nuovo, questo non ha alcun senso logico per un computer. Un'affermazione logica come questa descrive uno stato di cose in un istante nel tempo. Un giocatore non può fare una mossa in un gioco quando il gioco è finito.

Per evitare confusione, non giocare mai con la formulazione della logica. Usa sempre gli operatori corretti. Le tabelle di verità sono estremamente utili in caso di dubbio.

La logica e gli algoritmi trattano la forma "se-allora" in modo leggermente diverso l'uno dall'altro. Considera "se X, allora Y". Mentre un'implicazione logica afferma come i valori di verità di X e Y sono correlati, l'intento della forma algoritmica "se-allora" è di rendere l'esecuzione del passaggio Y dipendente dal fatto che X sia vera. Se X non è vera, il passaggio Y non viene eseguito.

Mancanza di alcune eventualità

Un'altra "insidia" relativa all'uso di "se-allora" negli algoritmi è cosa fare quando la condizione è falsa. Ad esempio, ecco un algoritmo utilizzato per valutare il lavoro di uno studente. È molto semplice. Uno studente può ottenere uno dei due voti (promosso o bocciato) e lo studente ha bisogno di un punteggio superiore al 50% per ottenere un voto di promozione (vedi Figura 2.6).

[Diagramma dei livelli di voto per bocciato e promosso]

Se il punteggio è maggiore del 50%, allora segna il voto dello studente come promosso.

Vedi un problema?

Cosa succede nei casi in cui lo studente ottiene un punteggio pari o inferiore al 50%? Il buon senso dice a un essere umano che lo studente dovrebbe ottenere un voto di bocciatura, perché questo è ciò che il problema implica. Ma ricorda, i computer non hanno buon senso e non faranno nulla che tu non dica loro di fare. Ecco perché gli algoritmi includono anche la forma "se-allora-altrimenti". Usando questo, potresti modificare l'algoritmo in questo modo:

se il punteggio è maggiore del 50%, allora segna il voto dello studente come promosso, altrimenti segna il voto dello studente come bocciato.

Quando la condizione in un "se-allora-altrimenti" è vera, viene eseguita solo la parte dopo "allora"; quando la condizione è falsa, viene eseguita solo la parte dopo "altrimenti".

Condizionali complessi

Di per sé, l'operatore NOT è semplice da capire. Ad esempio, l'algoritmo di valutazione potrebbe essere riscritto come segue:

se il punteggio non è maggiore del 50%, allora segna il voto dello studente come bocciato, altrimenti segna il voto dello studente come promosso.

Nessuna sorpresa. Tuttavia, la difficoltà può sorgere quando una condizione diventa più complessa. Immagina che venga introdotto un voto aggiuntivo: agli studenti che ottengono un punteggio superiore all'80% viene assegnato un "distinguished pass" (vedi Figura 2.7). Un "pass" ordinario verrebbe assegnato solo ai punteggi superiori a 50 e inferiori all'80%.

[Diagramma dei livelli di voto per bocciato, promosso ordinario e promosso con lode]

Per verificare un punteggio di promozione ordinario, potresti iniziare un "se-allora" che verifica due condizioni in questo modo:

se il punteggio non è inferiore al 51% o superiore all'80%, allora segna il voto dello studente come promosso ordinario ...

Un errore deliberato si trova in questo piccolo algoritmo. Tutto sarà rivelato a breve.

Ciò potrebbe sembrare preciso a prima vista, ma potrebbe essere interpretato in almeno due modi diversi. In particolare, il "non" si applica a entrambe le condizioni o solo alla prima? Prendere il linguaggio di tutti i giorni come guida suggerisce che si applichi a entrambi, proprio come dire "l'auto non è grande o rossa" significa che l'auto non è né grande né rossa. Tuttavia, i regni del linguaggio naturale e della logica non sempre si allineano. Questo è un caso in cui non lo fanno e vedremo perché a breve.

Per evitare tale confusione, la logica simbolica fornisce regole molto specifiche su come interpretare le varie parti delle affermazioni logiche. Queste regole sono chiamate ordine di precedenza. Esse dettano in quale ordine ciascuna parte di un'espressione logica deve essere valutata. Funziona proprio come in matematica.

Tabella 2.8 Ordine di precedenza per operatori logici selezionati

Rango	**Operatore**	**Nome**
1	( )	Parentesi
2	¬	Non
3	> <	Maggiore di Minore di
4	∧	AND logico
5	∨	OR logico

Esporta in Fogli

Quando si valuta qualcosa come

9 - 3 × 4

si ottengono risposte diverse a seconda dell'ordine in cui si eseguono la sottrazione e la moltiplicazione. Ma a rigor di termini, c'è solo una risposta corretta. Questo perché le regole dell'ordine di precedenza dettano in quale ordine ciascuna parte viene valutata; quelle con precedenza maggiore vengono eseguite per prime. Per rendere esplicito l'ordine, puoi mettere le parentesi attorno a ciascuna espressione. Se dovessimo metterle esplicitamente, sarebbe così:

Prima la moltiplicazione 9 - (3 × 4) poi la sottrazione (9 - (3 × 4))

Tecnicamente, ciò significa esattamente la stessa cosa dell'espressione originale. Ora possiamo vedere chiaramente che la moltiplicazione deve avvenire per prima e quindi la risposta corretta è -3. Se volessi eseguire prima la sottrazione, andrebbe bene, ma dovresti inserire le tue parentesi nell'espressione. In questo caso, apparirebbe così:

(9 - 3) × 4

Questo cambia il significato dell'espressione e il suo risultato (24).

Torniamo al nostro esempio di valutazione dello studente. Prima di applicare le regole, dovremmo riscrivere l'espressione usando la notazione corretta. Prendendo ogni condizione una per una:

il punteggio non è inferiore al 50%

diventa

¬ punteggio < 51

e maggiore dell'80% diventa punteggio > 80. Collegandoli con il simbolo OR appropriato otteniamo:

se ¬punteggio < 51 ∨ punteggio > 80

Per aiutarci a vedere l'ordine in cui le cose vengono valutate, mettiamo le parentesi implicite al loro posto. La negazione ha la precedenza più alta degli operatori qui, quindi ottiene prima le parentesi:

se ¬(punteggio < 51) ∨ punteggio > 80

I confronti hanno una precedenza maggiore delle disgiunzioni, quindi ottengono le loro parentesi successive (il confronto a sinistra ha già le parentesi grazie all'operatore di negazione):

se ¬(punteggio < 51) ∨ (punteggio > 80)

Infine, la disgiunzione ottiene le sue parentesi.

se (¬(punteggio < 51) ∨ (punteggio > 80))

L'espressione è pronta per essere valutata. Per farlo, dobbiamo assegnare un valore a "punteggio". Scegliamo arbitrariamente 55. Ora che le parentesi sono al loro posto, possiamo andare da sinistra a destra, valutando ogni parte man mano che procediamo. NOT inverte il valore di verità della cosa alla sua destra, quindi dobbiamo capire se quella cosa è vera o meno. 55 non è minore di 51, quindi sostituiamo "punteggio < 51" con il suo valore di verità:

se ¬(falso) ∨ (punteggio > 80)

In altre parole:

se vero ∨ (punteggio > 80)

Abbiamo elaborato il valore della condizione a sinistra di OR. Ora dobbiamo valutare la condizione alla sua destra. 55 non è maggiore di 80, quindi:

se vero ∨ falso

Consultando la tabella di verità per OR, scopriamo che quando una o entrambe le condizioni sono vere, l'intera espressione è vera. In questo caso, lo studente ottiene correttamente un "pass" ordinario.

Proviamo un altro punteggio per assicurarci che l'espressione funzioni anche per i "distinguished pass". Useremo 85 come punteggio. 85 non è minore di 51, quindi:

se ¬(falso) ∨ (punteggio > 80)

In altre parole:

se vero ∨ (punteggio > 80)

E 85 è maggiore di 80, quindi:

se vero ∨ vero

Di nuovo, abbiamo valutato l'espressione e abbiamo scoperto che è vera. Ma questo non è ciò che volevamo. Guarda di nuovo l'espressione nel contesto dell'algoritmo:

se il punteggio non è inferiore al 51% o superiore all'80%, allora segna il voto dello studente come "pass" ordinario, ...

Abbiamo dimostrato che, secondo l'algoritmo nel suo stato attuale, lo studente ottiene un "pass" ordinario anche se ha ottenuto un punteggio superiore a 80! Il problema è che il nostro modo di esprimere le nostre intenzioni nel linguaggio naturale non concorda del tutto con le regole della logica. Per correggere il nostro algoritmo originale, dovremmo inserire le nostre parentesi esplicite nell'espressione per "sovrascrivere" i soliti ordini di precedenza, in questo modo:

se ¬(punteggio < 51 ∨ punteggio > 80)

Questa volta, valutiamo prima tutto ciò che è all'interno delle parentesi prima di valutare la negazione. Ora, per un punteggio di 85, la procedura è simile a questa:

se ¬(punteggio < 51 ∨ punteggio > 80) se ¬(vero ∨ punteggio > 80) se ¬(vero ∨ vero) se ¬(vero) se falso

Lo studente quindi non riceverà il voto sbagliato, il che è positivo perché è sempre imbarazzante quando succede.

.